

Guia Rápido de Git para Projetos Colaborativos

Este guia é uma referência simples para usares o Git em projetos colaborativos. Inclui passos essenciais, comandos úteis e boas práticas para trabalhares com colegas de forma eficiente.

1 Configuração Inicial

- **Instala o Git:** Baixa em <https://git-scm.com/> (Windows) ou usa `brew install git` (Mac).
- **Configura o Git:** Define o teu nome e email:

```
git config --global user.name "Teu Nome"
git config --global user.email "teu.email@exemplo.com"
```

- **Cria uma conta no GitHub:** Vai a <https://github.com/>.

2 Criando e Clonando um Repositório

- **Cria um repositório no GitHub:**
 1. Inicia sessão no GitHub.
 2. Clica em “New” e dá um nome ao repositório.
 3. Adiciona um README.md e cria.
- **Convida colaboradores:** Em “Settings” > “Collaborators”, adiciona os usernames.
- **Clona o repositório localmente:**

```
git clone <URL-do-repositório>
cd <nome-da-pasta>
```

3 Fluxo de Trabalho com Branches

- **Nunca trabalhes na main!** Usa branches para cada funcionalidade.
- **Cria uma branch:**

```
git checkout -b feature/nome-da-funcionalidade
```

- **Verifica a branch atual:** `git branch` (a atual tem um *).
- **Edita ficheiros:** Usa o VS Code ou outro editor.
- **Adiciona mudanças:**

```
git add <ficheiro> % ou git add . para todos
```

- **Faz commit:**

```
git commit -m "Mensagem clara do que foi feito"
```

- **Envia para o GitHub:**

```
git push origin feature/nome-da-funcionalidade
```

4 Pull Requests e Merge

- **Cria um Pull Request (PR):**

1. No GitHub, vai a “Pull Requests” > “New Pull Request”.
2. Escolhe a base (`main`) e a branch a comparar.
3. Dá um título claro e descrição.

- **Revisa o PR:** Verifica o código com colegas antes de fazer merge.
- **Faz merge:** Clica em “Merge pull request” e confirma.
- **Atualiza localmente:**

```
git checkout main  
git pull origin main
```

5 Resolvendo Conflitos

- Conflitos ocorrem quando duas branches alteram a mesma linha.
- Para resolver:

```
git checkout main  
git merge feature/nome-da-branch
```

- O Git marca conflitos com «» e «». Edita o ficheiro manualmente, depois:

```
git add <ficheiro>  
git commit
```

6 Comandos Úteis

- `git status`: Mostra o estado atual (ficheiros alterados, branch).
- `git log`: Vê o histórico de commits.
- `git fetch origin`: Atualiza informações do remoto sem mudar local.
- `git pull origin main`: Puxa atualizações da `main`.
- `git branch -d feature/nome`: Apaga uma branch local após merge.

7 Boas Práticas

- Usa nomes de branches descritivos: `feature/nome`, `bugfix/nome`.
- Faz commits pequenos e com mensagens claras.
- Sempre revisa PRs antes de merge.
- Usa `.gitignore` para ignorar ficheiros desnecessários (ex: `node_modules/`).
- Antes de começar, faz `git pull` para evitar conflitos.

8 Dicas Extras

- Usa o VS Code com extensões como “GitHub Pull Requests and Issues”.
- Para ajuda: `git -help` ou pesquisa online.
- Pratica com projetos pequenos para ganhar confiança!